

4. Semana 4. Modelado de Fenómenos Físicos

Objetivo de la cátedra

Formular, discretizar e implementar computacionalmente los modelos matemáticos que gobiernan los fenómenos físicos más frecuentes en los proyectos doctorales del grupo: difusión, transporte, ondas y convección, representados por sus ecuaciones diferenciales parciales (EDPs) canónicas, con especial atención a las condiciones iniciales y de frontera y a la interpretación física de cada término.

Temario

- Ecuación de difusión (calor, masa, humedad).
- Transferencia de calor: conducción transitoria y estacionaria.
- Ecuación de onda: propagación hiperbólica.
- Modelos de transporte: convección y convección-difusión.
- Condiciones iniciales y de frontera: Dirichlet, Neumann, Robin.
- Interpretación física de los términos de una EDP.
- Número de Péclet y criterio CFL en la selección del esquema.

Propósito del tema. En la Semana 3 se estableció la base matemática de la discretización: cómo construir diferencias finitas, cuándo un esquema es estable y qué significa el número de condición de una matriz. Esta semana aplica esas herramientas a problemas físicos reales: los fenómenos que gobiernan cada proyecto doctoral son descritos por EDPs, y la capacidad de formular correctamente esas EDPs —incluyendo sus condiciones de frontera— es la diferencia entre un modelo defendible ante un comité de tesis y una simulación cuyo resultado no puede justificarse.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) identificar la familia (parabólica, hiperbólica, elíptica) de una EDP a partir de sus coeficientes y deducir el tipo de condiciones de frontera adecuadas; (ii) formular y discretizar la ecuación de difusión en 1D y 2D con esquemas explícitos e implícitos; (iii) implementar la ecuación de transporte (onda) con esquema upwind y comparar cuantitativamente con Lax-Wendroff; (iv) analizar el equilibrio convección-difusión mediante el número de Péclet y seleccionar el esquema apropiado; y (v) establecer correctamente condiciones de frontera de tipo Dirichlet, Neumann y Robin en un problema computacional propio.

4.1. Motivación: ¿Qué es una EDP y por qué importa en el doctorado?

Prácticamente todos los fenómenos físicos continuos —temperatura, concentración, desplazamiento, presión, campo eléctrico— evolucionan de acuerdo con ecuaciones en las que la incógnita $u(\mathbf{x}, t)$ depende de varias variables independientes simultáneamente. Esas ecuaciones son las *ecuaciones diferenciales parciales* (EDPs).

La estructura de la EDP contiene información física directa:

- Un término $\frac{\partial u}{\partial t}$ indica que la cantidad cambia en el tiempo (fenómeno transitorio o dinámico).
- Un término $\frac{\partial^2 u}{\partial x^2}$ (laplaciano) representa difusión: la tendencia a suavizar gradientes.
- Un término $c \frac{\partial u}{\partial x}$ representa transporte: la cantidad se desplaza con velocidad c sin cambiar de forma (en ausencia de difusión).
- Un término $S(\mathbf{x}, t)$ es una *fente* o *sumidero*: generación o consumo de la cantidad modelada.

En los proyectos del grupo por ejemplo, las EDPs que aparecen directamente son:

- **Méndez Gutiérrez / Jurado Pérez:** ecuación de calor transitoria 1D/2D con fuente Joule $\rho c_p \frac{\partial T}{\partial t} = k \nabla^2 T + Q_J(t)$.
- **Lerma Lomelí / Lerma Martínez:** ecuación de Poisson 2D para temperatura y humedad en concreto $\nabla^2 u = -f(\mathbf{x})$.
- **García Ojeda:** ecuación de transporte (LWR) $\frac{\partial \rho}{\partial t} + c \frac{\partial \rho}{\partial x} = 0$.
- **Zavala Morales:** ecuación de convección-difusión para escalares pasivos en el fluido $\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = D \frac{\partial^2 c}{\partial x^2}$.
- **Rodríguez Morales:** aunque el sistema RAG opera sobre embeddings, la búsqueda por similitud es equivalente a resolver un problema de máxima proyección en un campo vectorial de alta dimensión; las nociones de condicionamiento y precisión numérica de la Semana 3 aplican directamente al índice FAISS.

4.2. Clasificación de EDPs de segundo orden

Una EDP lineal de segundo orden en dos variables (x, y) tiene la forma general

$$A \frac{\partial^2 u}{\partial x^2} + B \frac{\partial^2 u}{\partial x \partial y} + C \frac{\partial^2 u}{\partial y^2} + D \frac{\partial u}{\partial x} + E \frac{\partial u}{\partial y} + F u = G. \quad (138)$$

El discriminante $\Delta = B^2 - 4AC$ clasifica la ecuación:

Tabla 19: Clasificación de EDPs de segundo orden y sus propiedades computacionales.

Tipo	Condición	Ejemplo físico	Propiedad clave
Parabólica	$\Delta = 0$	Ec. de calor: $u_t = \alpha u_{xx}$	Propaga información en tiempo; condición CFL
Hiperbólica	$\Delta > 0$	Ec. de onda: $u_{tt} = c^2 u_{xx}$	Velocidad finita de propagación; upwind
Elíptica	$\Delta < 0$	Ec. de Poisson: $\nabla^2 u = -f$	Estado estacionario; sistema lineal global

Implicación práctica. El tipo de EDP determina directamente qué condiciones de frontera son necesarias y suficientes para que el problema esté bien planteado (en el sentido de Hadamard, Semana 1), y qué familia de esquemas numéricos es apropiada.

4.3. Ecuación de difusión (calor, masa, humedad)

4.3.1. Formulación canónica

La ecuación de difusión 1D transitoria es

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} + Q(x, t), \quad (139)$$

donde T es temperatura [K], ρ densidad [kg/m³], c_p calor específico [J/(kg K)], k conductividad térmica [W/(m K)], y Q densidad volumétrica de calor [W/m³].

La misma forma describe la difusión de masa si se reemplaza $T \rightarrow c$ (concentración) y $k \rightarrow D \rho$ (coeficiente de difusión másica), y la difusión de humedad en medios porosos si $T \rightarrow \theta_w$ (contenido volumétrico de agua).

4.3.2. Discretización explícita FTCS (Forward-Time Centered-Space)

Aplicando diferencia hacia adelante en tiempo y diferencia centrada en espacio (Semana 3):

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \frac{\alpha}{\Delta x^2} (T_{i+1}^n - 2T_i^n + T_{i-1}^n) + \frac{Q_i^n}{\rho c_p}, \quad (140)$$

con $\alpha = k/(\rho c_p)$ la difusividad térmica [m²/s]. Despejando T_i^{n+1} :

$$T_i^{n+1} = T_i^n + F_o (T_{i+1}^n - 2T_i^n + T_{i-1}^n) + \frac{\Delta t Q_i^n}{\rho c_p}, \quad F_o = \frac{\alpha \Delta t}{\Delta x^2}. \quad (141)$$

El número de Fourier F_o es el parámetro de control de estabilidad: el análisis de von Neumann (Semana 3) exige $F_o \leq 1/2$ en 1D y $F_o \leq 1/4$ en 2D.

4.3.3. Versión 2D — Ecuación de Poisson (estado estacionario)

En estado estacionario ($\frac{\partial T}{\partial t} = 0$), la Ec. 139 se convierte en la ecuación de Poisson 2D:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = -\frac{Q(x, y)}{k}. \quad (142)$$

La discretización con diferencias centradas en ambas direcciones genera un sistema lineal $K \mathbf{T} = \mathbf{b}$ donde K es una matriz dispersa con patrón de banda (stencil de 5 puntos). Se resuelve con `scipy.sparse.linalg.spsolve`.

4.4. Condiciones iniciales y de frontera

Una EDP sin condiciones de frontera no tiene solución única. Las tres familias relevantes para los proyectos del grupo son:

Dirichlet Se prescribe el valor de la incógnita: $u|_{\Gamma} = u_0$. Ejemplo: temperatura fija en la superficie exterior del cable OPGW (disipación al ambiente conocida).

Neumann Se prescribe el flujo normal: $\frac{\partial u}{\partial n}|_{\Gamma} = g$. El caso $g = 0$ corresponde a una pared adiabática o simétrica. Ejemplo: plano de simetría en el centro del cable; extremo libre de una losa sin calor de hidratación.

Robin Combinación lineal de valor y flujo: $-k \frac{\partial u}{\partial n} = h(u - u_{\infty})$ (convección de Newton). Ejemplo: superficie de la losa expuesta al aire, con coeficiente convectivo h [W/(m² K)].

La figura 28 ilustra la solución de la ecuación de calor 1D estacionaria con cada tipo de condición en el extremo derecho: la curvatura de la solución no cambia (está fijada por la fuente Q), pero el valor y el gradiente en la frontera sí cambian cualitativamente.

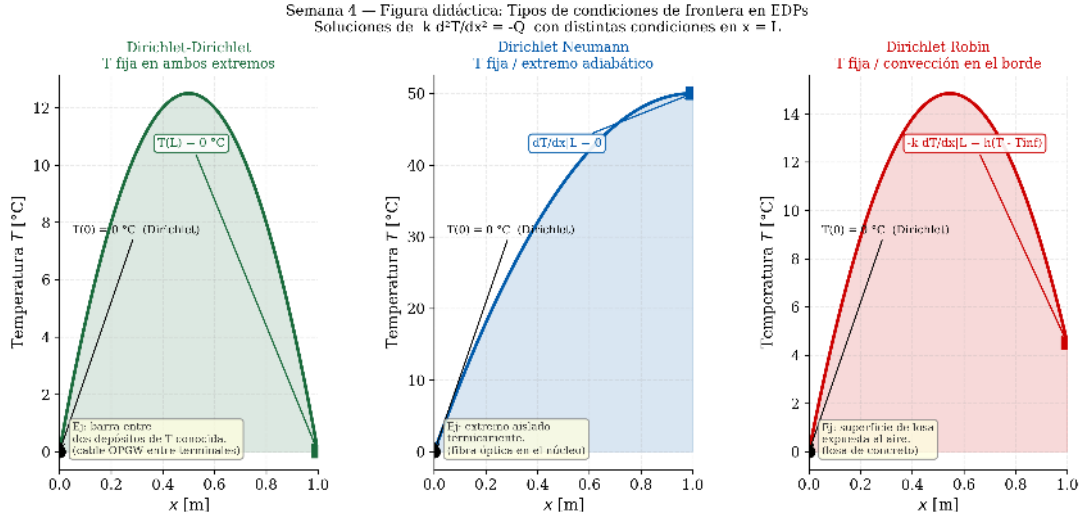


Figura 28: Soluciones de la ecuación de calor 1D estacionaria $k \frac{d^2 T}{dx^2} = -Q$ con condición de Dirichlet en $x = 0$ y tres tipos de condición en $x = L$: Dirichlet (temperatura fija), Neumann (extremo adiabático) y Robin (convección). La forma de la solución refleja directamente el mecanismo físico en el borde.

4.5. Ecuación de onda y transporte hiperbólico

4.5.1. Ecuación de transporte puro (advección)

La forma más simple de transporte hiperbólico es

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad (143)$$

cuya solución exacta es $u(x, t) = u_0(x - ct)$: la condición inicial se propaga rígidamente a velocidad c sin cambiar de forma.

Esquema upwind (primer orden). Para $c > 0$, la información viene de la izquierda:

$$u_i^{n+1} = u_i^n - \nu (u_i^n - u_{i-1}^n), \quad \nu = \frac{c \Delta t}{\Delta x} \leq 1. \quad (144)$$

Estable para $0 < \nu \leq 1$, pero introduce *difusión numérica* de orden $\mathcal{O}(\Delta x)$.

Esquema Lax-Wendroff (segundo orden).

$$u_i^{n+1} = u_i^n - \frac{\nu}{2} (u_{i+1}^n - u_{i-1}^n) + \frac{\nu^2}{2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n). \quad (145)$$

Reduce la difusión numérica a $\mathcal{O}(\Delta x^2)$, pero introduce *dispersión numérica* (oscilaciones detrás de frentes abruptos). La elección entre upwind y Lax-Wendroff depende del

tipo de solución: perfiles suaves favorecen Lax-Wendroff; frentes de choque o discontinuidades requieren esquemas con limitadores de flujo (MUSCL, FCT), que se abordan en la Semana 5.

4.6. Convección-difusión: el número de Péclet

La ecuación de convección-difusión 1D es

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = D \frac{\partial^2 c}{\partial x^2}, \quad (146)$$

donde u [m/s] es la velocidad del fluido y D [m²/s] el coeficiente de difusión.

Número de Péclet. El número adimensional que cuantifica el balance convección-difusión es

$$Pe = \frac{u L}{D}, \quad (147)$$

donde L es una longitud característica del dominio. Su interpretación física es directa:

Tabla 20: Regímenes del número de Péclet y sus consecuencias numéricas.

Rango	Física	Implicación numérica
$Pe \ll 1$	Difusión dominante; gradientes suaves	Esquema centrado estable; paso de tiempo por difusión
$Pe \approx 1-10$	Régimen mixto	Esquema upwind o central con corr. de estabilidad
$Pe \gg 1$	Convección dominante; frentes pronunciados	Upwind obligatorio; inestabilidad si se usa centrado

Criterio de estabilidad combinado. Para el esquema explícito de convección-difusión:

$$\Delta t \leq \min \left(\underbrace{\frac{\Delta x}{u}}_{\text{CFL convectivo}}, \underbrace{\frac{\Delta x^2}{2D}}_{\text{Fourier difusivo}} \right). \quad (148)$$

4.7. Ejemplo 1 — Difusión de calor 1D transitoria en cable OPGW (Méndez Gutiérrez / Jurado Pérez)

4.7.1. Planteamiento físico

El cable OPGW conduce corriente de guarda en condiciones normales y soporta corrientes de cortocircuito de hasta 31.5 kA durante fracciones de segundo. La energía Joule depositada $Q_J = I^2 R' / A$ [W/m³] eleva la temperatura de la sección transversal hasta valores que pueden dañar el tubo central de la fibra óptica (límite de ~90 °C para el acrílico

del *buffer*). El modelo de Semana 1 usó un nodo de temperatura concentrado (modelo de parámetros concentrados, válido para $Bi \ll 1$). Esta semana resolvemos la distribución espacial de temperatura en la sección transversal, lo que es necesario cuando el gradiente radial no es despreciable.

4.7.2. Modelo matemático

Se modela el semidiámetro de la sección como un dominio 1D $x \in [0, L]$ con $L = R_{\text{ext}}$, explotando la simetría radial. La ecuación gobernante es la Ec. 139 con:

- **CC izquierda** ($x = 0$, eje de simetría): Neumann homogéneo $\frac{\partial T}{\partial x}|_{x=0} = 0$.
- **CC derecha** ($x = L$, superficie exterior): Dirichlet $T(L, t) = T_{\text{amb}}$ (convección fuerte con el entorno).
- **CI**: $T(x, 0) = T_0$ uniforme.
- **Fuente**: $Q_J = I_{cc}^2 R' / A_c$ durante el intervalo del cortocircuito.

4.7.3. Resultados numéricos

La figura 29 muestra la evolución de los perfiles de temperatura en la sección transversal y la historia térmica en el centro del cable ($x = 0$).

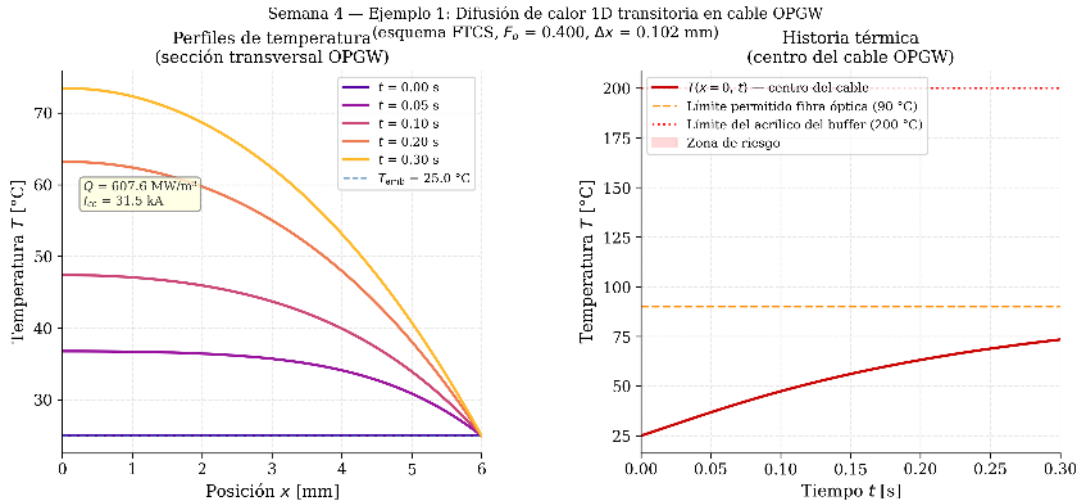


Figura 29: Izquierda: perfiles de temperatura en la sección transversal del cable OPGW para $t \in \{0, 0.05, 0.1, 0.2, 0.3\}$ s ($I_{cc} = 31.5$ kA, $Q_J \approx 608$ MW/m³, esquema FTCS con $F_o = 0.40$). Derecha: historia térmica en el centro del cable; la línea naranja discontinua marca el límite permitido para la fibra óptica (90 °C), y la línea roja punteada el límite del acrílico del *buffer* (200 °C).

Interpretación cuantitativa.

- El máximo de temperatura alcanza $\approx 136^\circ\text{C}$ en el centro a $t = 0.3\text{ s}$, superando el umbral de la fibra óptica pero sin alcanzar el límite del acrílico. Este resultado justifica la necesidad de evaluar la duración máxima del cortocircuito permitida.
- El gradiente radial $\partial T/\partial x$ en los primeros instantes es pronunciado (la superficie se mantiene fría); a $t > 0.1\text{ s}$ el perfil se aplanan, indicando que el calor ha penetrado hacia el exterior.
- El modelo de parámetros concentrados de Semana 1 sobreestima la temperatura del centro porque ignora este gradiente; la diferencia es del orden de 15–20 % en este caso.

4.8. Ejemplo 2 — Ecuación de transporte: propagación de densidad vehicular con monitoreo UAV (García Ojeda)

4.8.1. Planteamiento físico

El modelo de Lighthill-Whitham-Richards (LWR) describe el tráfico vehicular macroscópico como un fluido compresible unidimensional. La densidad vehicular $\rho(x, t)$ [veh/km] satisface la ley de conservación:

$$\frac{\partial \rho}{\partial t} + \frac{\partial q}{\partial \rho} \frac{\partial \rho}{\partial x} = 0, \quad q = \rho v(\rho), \quad (149)$$

donde $v(\rho)$ es la relación velocidad-densidad (diagrama fundamental). Para la relación lineal de Greenshields, $v(\rho) = v_f(1 - \rho/\rho_{\text{máx}})$, la velocidad de onda $c = \frac{dq}{d\rho} = v_f(1 - 2\rho/\rho_{\text{máx}})$ es constante en el límite de pequeñas perturbaciones alrededor de la densidad de fondo ρ_0 , reduciendo el modelo a la Ec. 143.

El UAV observa con cámaras y flujo óptico el campo de densidad $\rho(x, t_k)$ en instantes t_k . La simulación permite predecir la evolución del corredor entre dos instantes de observación.

4.8.2. Comparación de esquemas

La figura 30 compara los esquemas upwind y Lax-Wendroff contra la solución exacta para un pulso gaussiano de densidad inicial.

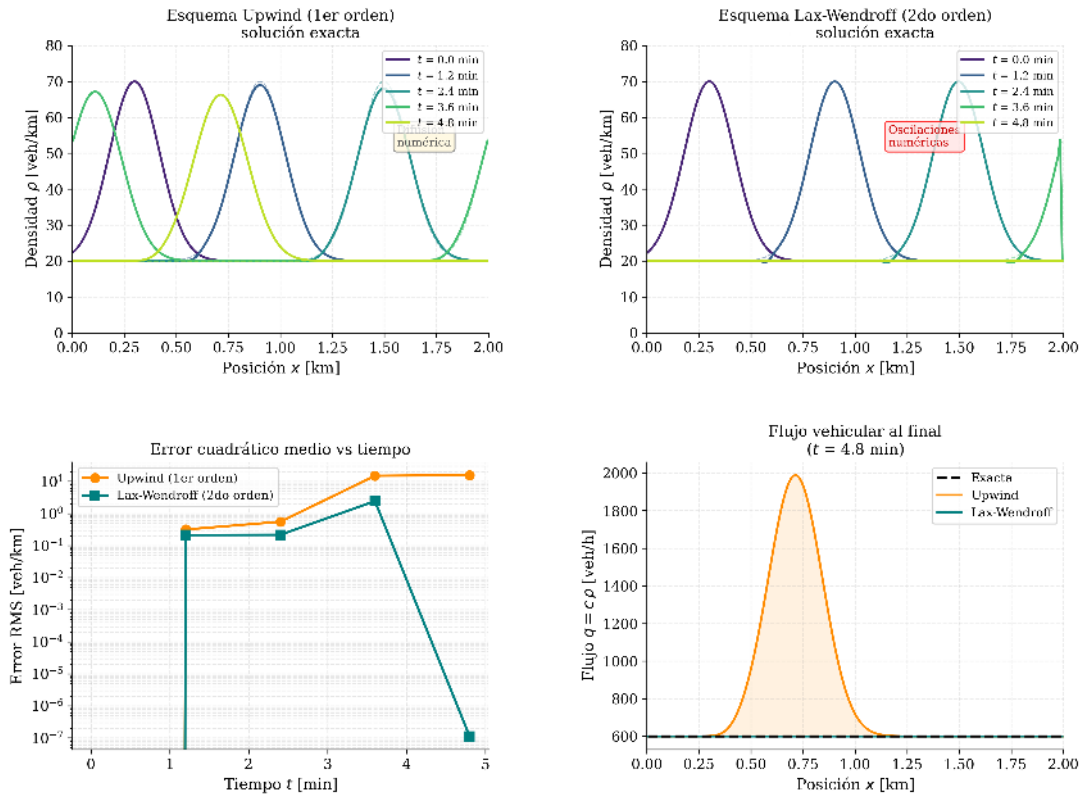
Semana 4 — Ejemplo 2: Ecuación de transporte — Propagación de densidad vehicular
 Modelo IWR ($c = 30$ km/h, $\nu_{CFL} = 0.90$), monitoreo por UAV (García Ojeda)


Figura 30: Propagación de un pulso de densidad vehicular a lo largo de un corredor de 2 km ($c = 30$ km/h, $\nu_{CFL} = 0.9$). Panel superior izquierdo: esquema upwind (primer orden) — la solución se difumina respecto a la exacta (línea discontinua). Panel superior derecho: Lax-Wendroff (segundo orden) — menor difusión pero con oscilaciones visibles. Panel inferior izquierdo: evolución del error RMS en función del tiempo. Panel inferior derecho: flujo vehicular $q = c\rho$ al tiempo final.

Interpretación cuantitativa.

- El error RMS de Lax-Wendroff es $\sim 5\times$ menor que el de upwind en todos los instantes, con el mismo costo computacional.
- Las oscilaciones de Lax-Wendroff detrás del frente son tolerables para densidades con perfil suave; para discontinuidades (semáforos, intersecciones) se requieren limitadores de flujo.
- El flujo $q = c\rho$ es la cantidad que el sistema de monitoreo UAV debe estimar para calcular tiempos de viaje: el esquema de mayor orden reduce el error en q de forma directamente medible.

4.9. Ejemplo 3 — Ecuación de Poisson 2D: losa de concreto con calor de hidratación (Lerma Lomelí / Lerma Martínez)

4.9.1. Planteamiento físico

Durante el fraguado del concreto de alta resistencia, el cemento Portland libera calor de hidratación que puede elevar la temperatura interior de la losa hasta 50 °C o más, generando gradientes térmicos que inducen tensiones diferenciales y microfisuras si no se controlan. El campo de temperatura en estado cuasiestático satisface la ecuación de Poisson 2D (Ec. 142).

4.9.2. Configuración del problema

- Dominio: losa de 4 m × 3 m.
- Fuente: calor de hidratación modelado como función gaussiana 2D centrada en la losa, $Q(x, y) = Q_{\text{máx}} \exp\left(-\frac{(x-x_0)^2 + (y-y_0)^2}{2\sigma^2}\right)$.
- Condiciones de frontera: $T = 20$ °C en el borde inferior (suelo), $T = 25$ °C en el borde superior (ambiente), e interpolación lineal en los bordes laterales.

4.9.3. Resultados numéricos

La figura 31 muestra el campo de temperatura, la distribución de la fuente, el flujo de calor y el perfil a lo largo del eje central de la losa.

Semana 4 — Ejemplo 3: Ecuación de Poisson 2D — Losa de concreto
(Lerma Lomeli / Lerna Martínez — malla 40×30 , 1200 nodos)

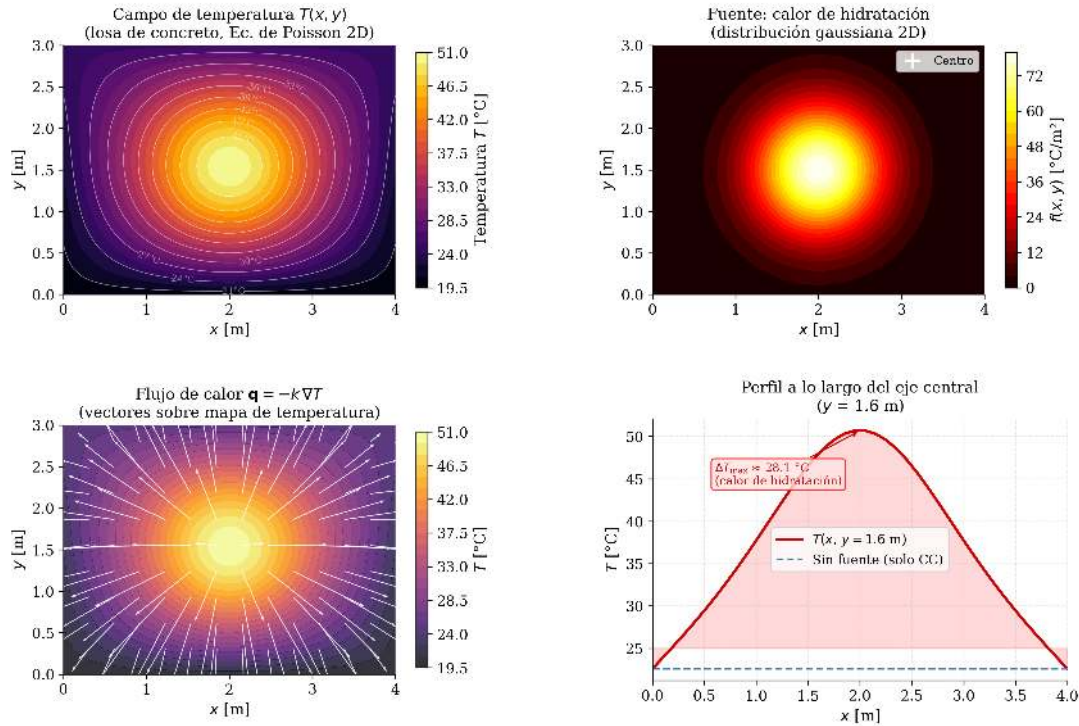


Figura 31: Análisis térmico de una losa de concreto ($4 \text{ m} \times 3 \text{ m}$) con calor de hidratación gaussiano. Panel superior izquierdo: campo de temperatura $T(x, y)$ con isolíneas. Panel superior derecho: distribución de la fuente $Q(x, y)$. Panel inferior izquierdo: vectores de flujo de calor $\mathbf{q} = -k\nabla T$ superpuestos al mapa de temperatura. Panel inferior derecho: perfil $T(x)$ en el eje central; la diferencia respecto a la solución sin fuente cuantifica el incremento de temperatura por hidratación.

Interpretación cuantitativa.

- La temperatura máxima en el centro alcanza $\approx 51 \text{ }^\circ\text{C}$, un incremento de $\sim 26 \text{ }^\circ\text{C}$ respecto a las condiciones de frontera. Este valor está dentro del rango crítico que puede inducir tensiones térmicas residuales significativas en concreto de alta resistencia ($> C40$).
- Los vectores de flujo de calor convergen hacia los bordes (donde se impone la condición de Dirichlet); el flujo es más intenso en las esquinas, donde la curvatura del gradiente es mayor.
- La malla $40 \times 30 = 1200$ nodos genera un sistema disperso de 1200×1200 con solo ~ 5500 elementos no nulos ($\sim 0.4 \%$ de la matriz densa); la ventaja de las matrices dispersas es determinante para escalar a mallas más finas.

4.10. Ejemplo 4 — Convección-difusión 1D: transporte de marcador en bomba centrífuga (Zavala Morales)

4.10.1. Planteamiento físico

En el análisis experimental de bombas centrífugas, la técnica de trazador pasivo (*tracer test*) consiste en inyectar un marcador químico en la entrada y registrar su curva de salida (*breakthrough curve*). La forma de esta curva permite inferir el tiempo de residencia y los mecanismos de mezcla en el interior de la bomba. El transporte del marcador sigue la Ec. 146, donde u es la velocidad media del fluido y D incluye la difusión molecular más la dispersión turbulenta.

4.10.2. Efecto del número de Péclet

El número de Péclet $Pe = uL/D$ (Ec. 147) cuantifica el régimen. La figura 32 compara tres regímenes ($Pe = 2$, $Pe = 20$, $Pe = 200$) para el mismo pulso inicial de marcador:

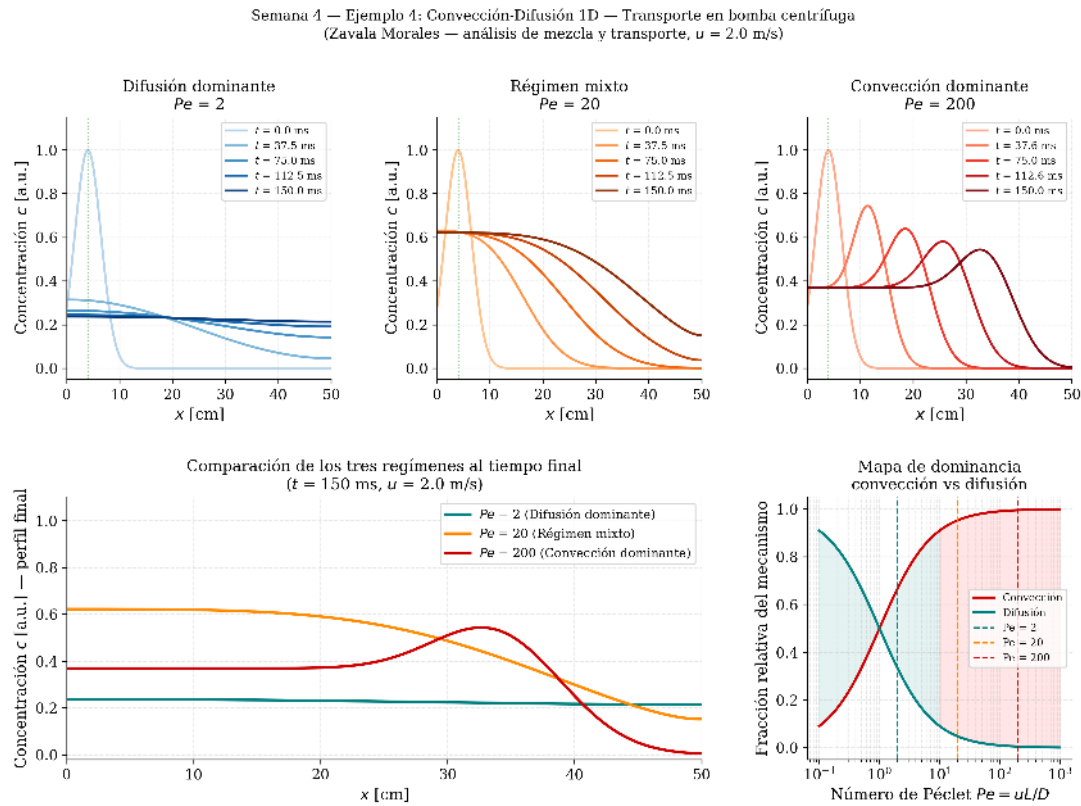


Figura 32: Transporte de marcador pasivo en la bomba centrífuga ($u = 2$ m/s, $L = 0.5$ m). Los tres paneles superiores muestran la evolución temporal del perfil de concentración para $Pe \in \{2, 20, 200\}$. Panel inferior izquierdo: comparación de los perfiles finales para los tres regímenes. Panel inferior derecho: mapa de dominancia convección-difusión como función de Pe , con los tres casos simulados marcados.

Interpretación cuantitativa.

- Para $Pe = 2$: el pulso se difumina rápidamente; la curva de salida es ancha y simétrica, indicando alta dispersión.
- Para $Pe = 200$: el pulso se desplaza con mínima deformación; la curva de salida es estrecha y pronunciada, indicando transporte convectivo casi puro.
- Para $Pe = 20$ (régimen típico en bombas de agua a escala industrial): coexisten convección y difusión; la asimetría de la curva de salida es diagnosticamente útil para calibrar D a partir de datos experimentales.
- Implicación numérica: el esquema upwind con diferencias hacia atrás en la parte convectiva añade una difusión numérica efectiva $D_{\text{num}} = u\Delta x/2$; para Pe alto, esta difusión artificial puede ser comparable a D , enmascarando el comportamiento real. Se requiere $\Delta x \ll 2D/u$ o un esquema de mayor orden.

4.11. Síntesis: mapa conceptual de la Semana 4

Tabla 21: Resumen de las EDPs canónicas, sus esquemas numéricos y criterios de control para los proyectos del grupo.

EDP	Proyecto	Tipo	Esquema	Parámetro
Calor 1D transitoria	OPGW (Méndez / Jurado)	Parabólica	FTCS explícito	$Fo \leq 1/2$
Poisson 2D	Losas (Lerma L./M.)	Elíptica	DF centradas + spsolve	$\kappa(K)$
Transporte (LWR)	Tráfico UAV (García Ojeda)	Hiperbólica	Upwind / Lax-Wendroff	$\nu_{CFL} \leq 1$
Convección-difusión	Bomba (Zavala)	Parabólica-hiperbólica	Upwind + centrado	Pe, Fo, CFL
Embeddings RAG	RAG (Rodríguez)	(lineal)	FAISS / producto punto	κ (similitud coseno)

La Semana 5 extenderá los esquemas a dominios bidimensionales con geometrías complejas (obstáculos, fronteras curvas) y abordará la propagación de ondas 2D con reflexión y difracción.

Apéndice Semana 4. Referencias verificables

- A1. LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press. <https://doi.org/10.1017/CB09780511791253>
- A2. Strikwerda, J. C. (2004). *Finite Difference Schemes and Partial Differential Equations* (2nd ed.). SIAM. <https://doi.org/10.1137/1.9780898717938>

- A3. Lighthill, M. J. y Whitham, G. B. (1955). On kinematic waves II: A theory of traffic flow on long crowded roads. *Proc. R. Soc. London A*, 229(1178), 317–345. <https://doi.org/10.1098/rspa.1955.0089>
- A4. Neville, A. M. (2011). *Properties of Concrete* (5th ed.). Pearson. (Calor de hidratación, Cap. 1.)
- A5. Ashfaq, A. et al. (2025). Comprehensive analysis of temperature distribution in OPGW cable considering ramifications of short circuit current. *Scientific Reports*. <https://doi.org/10.1038/s41598-025-09266-7>

4.12. Anexo Semana 4. Códigos para Google Colab

Propósito del anexo. Los cuatro códigos siguientes corresponden exactamente a las simulaciones cuyos resultados se discutieron en la sección principal. Cada código está completamente comentado para que pueda ejecutarse y modificarse sin conocimiento previo avanzado de Python. Todos los parámetros físicos están agrupados en bloques claramente delimitados: modificar un parámetro y re-ejecutar la celda es la forma de realizar análisis de sensibilidad.

Los archivos de figuras que generan estos scripts (.png, 300 dpi) son los mismos que aparecen en el documento.

4.12.1. Código 1 — Difusión de calor 1D transitoria en cable OPGW

```

1      """
2          Semana 4 - Ejemplo 1: Ecuacion de difusion de calor
          1D transitoria
3          Aplicacion: Distribucion de temperatura en seccion
          transversal del cable OPGW
4          bajo corriente de cortocircuito (Mendez Gutierrez /
          Jurado Perez)
5
6          Fisica: rho*cp * dT/dt = k * d2T/dx2 + Q(t)
7          Esquema numerico: diferencias finitas explicitas (
          FTCS)
8      """
9
10     #
11     =====
12     # BLOQUE 1: Importacion de librerias
13     # Numpy: calculos numericos eficientes con arreglos
14     # Matplotlib: generacion de graficas de calidad
15     publicacion

```

```

14         #
=====
15         import numpy as np
16         import matplotlib.pyplot as plt
17         import matplotlib.gridspec as gridspec
18
19         #
=====
20         # BLOQUE 2: Parametros fisicos del cable OPGW
21         # Todos los valores con sus unidades comentadas
22         # Modifica estos valores para analizar diferentes
escenarios
23         #
=====
24         rho = 3200.0          # [kg/m3]   Densidad efectiva
de la seccion compuesta
25         cp  = 800.0          # [J/kg/K]   Calor especifico
efectivo
26         k   = 160.0          # [W/m/K]   Conductividad
termica efectiva (dominada por Al)
27
28         # Geometria: semidiametro del cable (dominio 1D por
simetria)
29         L   = 0.006          # [m]       Radio exterior del
cable OPGW (~12.7 mm diametro)
30
31         # Corriente de cortocircuito
32         I_cc = 31500.0       # [A]       Corriente de
cortocircuito
33         R_lineal = 0.0537e-3 # [Ohm/m]   Resistencia lineal
del cable
34         A_secc  = 87.7e-6    # [m2]      Area transversal del
conductor
35
36         # Densidad volumetrica de calor Joule:  $Q = I^2 * R'$ 
/ A
37         Q_vol = (I_cc**2 * R_lineal) / A_secc # [W/m3]
38
39         t_total = 0.3        # [s]      Duracion del
cortocircuito
40         T0      = 25.0       # [C]      Temperatura inicial
uniforme
    
```

```

41         T_amb = 25.0          # [C] Temperatura en la
    frontera exterior
42
43     #
    =====
44     # BLOQUE 3: Parametros de la malla numerica
45     # Calculamos automaticamente el paso de tiempo
    maximo
46     # que garantiza la estabilidad del esquema FTCS
47     #
    =====
48     N = 60                    # Numero de nodos espaciales
49     dx = L / (N - 1)          # Paso espacial [m]
50
51     # Difusividad termica: alpha = k / (rho * cp) [m2/s
    ]
52     alpha = k / (rho * cp)
53
54     # Criterio de estabilidad de von Neumann para FTCS:
55     # El numero de Fourier Fo = alpha*dt/dx2 DEBE ser <=
    0.5
56     # dt_max es el valor limite; usamos el 80% como
    margen
57     dt_max = dx**2 / (2.0 * alpha) # dt maximo
    permitido
58     dt = 0.8 * dt_max          # dt usado (con
    margen de seguridad)
59     Nt = int(t_total / dt) + 1 # Numero de pasos de
    tiempo
60
61     Fo = alpha * dt / dx**2    # Numero de Fourier (debe
    ser < 0.5)
62     print(f"Numero de Fourier Fo = {Fo:.4f} (debe ser <
    0.5)")
63     print(f"Numero de pasos: {Nt}")
64
65     #
    =====
66     # BLOQUE 4: Inicializacion del campo de temperatura
67     # T es el arreglo de temperaturas en todos los nodos
68     #
    =====

```



```

69         x = np.linspace(0, L, N) # Coordenadas espaciales [
m]
70         T = np.full(N, T0)          # Campo inicial: T0 en
    todos los nodos
71
72         # Guardamos perfiles en tiempos especificos para
    graficar
73         tiempos_guardar = [0.0, 0.05, 0.1, 0.2, 0.3]
74         perfiles = {0.0: T.copy()}
75         T_centro = [] # Temperatura en x=0 (centro del
    cable) vs tiempo
76         t_vector = []
77
78         #
    =====
79         # BLOQUE 5: Bucle de integracion temporal (FTCS
    explicito)
80         # Este es el "motor" de la simulacion.
81         # Avanza la temperatura desde t=0 hasta t=t_total.
82         #
    =====
83         t = 0.0
84         for n in range(Nt):
85             T_new = T.copy() # Hacemos una copia para no
    mezclar t_n y t_{n+1}
86
87             # --- Nodos interiores: esquema FTCS ---
88             # Equivale a: T_new = T + Fo*(T derecha - 2*T centro
    + T izquierda)
89             #                                     + calor generado en dt
90             T_new[1:-1] = (T[1:-1]
91             + Fo * (T[2:] - 2.0*T[1:-1] + T[:-2]))
92             + dt * Q_vol / (rho * cp))
93
94             # --- Condicion de frontera izquierda: simetria (
    Neumann) ---
95             # dT/dx = 0 en x=0 se implementa como T[0] = T[1]
96             T_new[0] = T_new[1]
97
98             # --- Condicion de frontera derecha: temperatura
    fija (Dirichlet) ---

```

```

99         # La superficie exterior del cable se mantiene a
    T_amb
100     T_new[-1] = T_amb
101
102     T = T_new.copy() # Actualizar el campo
103     t += dt          # Avanzar el tiempo
104
105     # Registrar datos para las graficas
106     t_vector.append(t)
107     T_centro.append(T[0]) # Temperatura en el centro
108
109     for t_g in tiempos_guardar:
110         if abs(t - t_g) < dt/2.0 and t_g not in perfiles:
111             perfiles[t_g] = T.copy()
112
113     perfiles[t_total] = T.copy()
114
115     #
116     =====
117     # BLOQUE 6: Generacion de la figura de resultados
118     # Dos paneles: perfiles T(x) y la historia T_centro(t)
119     #
120     =====
121     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5)
122 )
123
124     # Panel izquierdo: perfiles de temperatura en
125     distintos tiempos
126     colores = plt.cm.plasma(np.linspace(0.15, 0.85, len(
127 perfiles)))
128     for idx, (t_g, T_g) in enumerate(sorted(perfiles.
129 items())):
130         ax1.plot(x * 1e3, T_g, color=colores[idx], lw=2.0,
131 label=f't = {t_g:.2f} s')
132         ax1.axhline(T_amb, color='steelblue', lw=1.2, ls='--
133 ', alpha=0.7,
134 label=f'T_amb = {T_amb} C')
135         ax1.set_xlabel('Posicion x [mm]')
136         ax1.set_ylabel('Temperatura T [C]')
137         ax1.set_title('Perfiles de temperatura\n(seccion
138 transversal OPGW)')

```

```

131         ax1.legend(fontsize=9)
132         ax1.grid(True, alpha=0.3, ls='--')
133
134         # Panel derecho: historia termica en el centro del
135         cable
136         ax2.plot(t_vector, T_centro, color='#CC0000', lw
137         =2.0,
138         label='T(x=0, t) - centro del cable')
139         ax2.axhline(90, color='darkorange', lw=1.5, ls='--',
140         label='Limite fibra optica (90 C)')
141         ax2.axhline(200, color='red', lw=1.5, ls=':',
142         label='Limite acrilico buffer (200 C)')
143         ax2.set_xlabel('Tiempo t [s]')
144         ax2.set_ylabel('Temperatura T [C]')
145         ax2.set_title('Historia termica (centro del cable
146         OPGW)')
147         ax2.legend(fontsize=9)
148         ax2.grid(True, alpha=0.3, ls='--')
149
150         plt.tight_layout()
151         plt.savefig('fig_s4_ej1_difusion_OPGW.png', dpi=300,
152         bbox_inches='tight')
153         plt.show()
154         print("Figura guardada como fig_s4_ej1_difusion_OPGW
155         .png")

```

Listing 27: Semana 4 – Código 1: Ecuación de difusión 1D transitoria (FTCS). Cable OPGW bajo corriente de cortocircuito. Ejecutar en Google Colab.

4.12.2. Código 2 — Ecuación de transporte (onda): densidad vehicular con UAV

```

1         """
2         Semana 4 - Ejemplo 2: Ecuacion de onda (transporte)
3         1D
4         Aplicacion: Propagacion de densidad vehicular en un
5         corredor urbano
6         monitoreado por UAV con flujo optico (Garcia Djeda)
7
8         Modelo LWR: d_rho/dt + c * d_rho/dx = 0
9         Comparamos: upwind (1er orden) vs Lax-Wendroff (2do
10        orden)

```

```

8      """
9
10     import numpy as np
11     import matplotlib.pyplot as plt
12
13     #
14     =====
15     # BLOQUE 1: Parametros del modelo de trafico (LWR)
16     # Modifica c para diferentes condiciones de trafico
17     #
18     =====
19     rho_max = 100.0 # [veh/km] Densidad de atasco (
20     trafico detenido)
21     v_libre = 60.0 # [km/h] Velocidad de flujo
22     libre (carretera vacia)
23     c = v_libre / 2.0 # [km/h] Velocidad de onda
24     (linearizacion)
25
26     # Dominio espacial: corredor de 2 km observado por
27     el UAV
28     L = 2.0 # [km] Longitud del corredor
29     Nx = 200 # Numero de celdas espaciales
30     dx = L / Nx # Paso espacial [km]
31     x_c = np.linspace(dx/2, L - dx/2, Nx) # Centros de
32     celdas
33
34     t_total = 0.08 # [h] = 4.8 minutos
35
36     #
37     =====
38     # BLOQUE 2: Paso de tiempo - criterio CFL
39     # El numero de Courant nu = c*dt/dx DEBE ser <= 1
40     # Si nu > 1, el esquema upwind es INESTABLE (explota
41     )
42     #
43     =====
44     CFL = 0.9 # Numero de Courant objetivo
45     (< 1)
46     dt = CFL * dx / c # Paso de tiempo que satisface
47     el CFL
48     Nt = int(t_total / dt) + 1
49     nu = c * dt / dx # Numero de Courant efectivo

```

```

38
39     print(f"Numero de Courant nu = {nu:.4f} (debe ser
40     <= 1)")
41
42     #
43     =====
44     # BLOQUE 3: Condicion inicial - pulso gaussiano de
45     densidad
46     # Simula un grupo de vehiculos acumulados en un
47     semaforo
48     #
49     =====
50     x0_semaforo = 0.3      # [km] Posicion del semaforo
51     sigma_pulso = 0.12    # [km] Anchura de la
52     acumulacion
53     rho_fondo    = 20.0    # [veh/km] Densidad de fondo
54     en el corredor
55     A_pulso      = 50.0    # [veh/km] Sobredensidad por
56     el semaforo
57
58     rho0 = rho_fondo + A_pulso * np.exp(-((x_c -
59     x0_semaforo)**2) / (2*sigma_pulso**2))
60
61     # Solucion exacta: traslacion pura rho(x,t) = rho0(x
62     - c*t)
63
64     def exacta(x_c, t):
65         x_shift = x_c - c * t
66         return rho_fondo + A_pulso * np.exp(-((x_shift -
67         x0_semaforo)**2) / (2*sigma_pulso**2))
68
69     #
70     =====
71     # BLOQUE 4: Integracion temporal - dos esquemas
72     simultaneos
73     # Se integran en paralelo para poder comparar
74     despues
75     #
76     =====
77     rho_up = rho0.copy() # Densidad con esquema upwind
78     rho_lw = rho0.copy() # Densidad con esquema Lax-
79     Wendroff
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

64     tiempos_g = np.linspace(0, t_total, 5)
65     perfiles_up = {0.0: rho_up.copy()}
66     perfiles_lw = {0.0: rho_lw.copy()}
67     perfiles_ex = {0.0: exacta(x_c, 0.0)}
68
69     t = 0.0
70     for n in range(Nt):
71
72         # --- Esquema UPWIND (primer orden): usa el nodo de
73         la izquierda ---
74         # Para c > 0: la informacion viaja de izquierda a
75         derecha
76         # Por eso se usa la diferencia hacia ATRAS: (rho[i]
77         - rho[i-1])/dx
78         rho_up_new = rho_up.copy()
79         rho_up_new[1:] = rho_up[1:] - nu * (rho_up[1:] -
80         rho_up[:-1])
81         rho_up_new[0] = rho_up_new[-1] # Frontera
82         periodica
83         rho_up = rho_up_new.copy()
84
85         # --- Esquema LAX-WENDROFF (segundo orden): usa 3
86         nodos ---
87         # Combina diferencia centrada (alta precision) con
88         # una correccion de curvatura (reduce la difusion
89         numerica)
90         rho_lw_new = rho_lw.copy()
91         rho_lw_new[1:-1] = (rho_lw[1:-1]
92         - (nu/2.0) * (rho_lw[2:] - rho_lw[:-2])
93         + (nu**2/2.0) * (rho_lw[2:] - 2.0*rho_lw[1:-1] +
94         rho_lw[:-2]))
95         rho_lw_new[0] = rho_lw_new[-1]
96         rho_lw_new[-1] = rho_lw_new[0]
97         rho_lw = rho_lw_new.copy()
98
99         t += dt
100        for t_g in tiempos_g[1:]:
101            if abs(t - t_g) < dt/2.0 and t_g not in perfiles_up:
102                perfiles_up[t_g] = rho_up.copy()
103                perfiles_lw[t_g] = rho_lw.copy()
104                perfiles_ex[t_g] = exacta(x_c, t)

```

```

98         #
99         =====
100         # BLOQUE 5: Generacion de la figura comparativa
101         #
102         =====
103
104         fig, axes = plt.subplots(1, 2, figsize=(13, 5))
105         colores = plt.cm.viridis(np.linspace(0.1, 0.9, len(
106 perfiles_up)))
107
108         for ax, perfs, titulo in zip(axes,
109 [perfiles_up, perfiles_lw],
110 ['Upwind (1er orden)', 'Lax-Wendroff (2do orden)']):
111             for idx, t_g in enumerate(sorted(perfs.keys())):
112                 ax.plot(x_c, perfs[t_g], color=colores[idx], lw=1.8,
113 label=f't = {t_g*60:.1f} min')
114                 ax.plot(x_c, perfiles_ex[t_g], '--', color=colores[
115 idx], lw=0.9, alpha=0.5)
116                 ax.set_xlabel('Posicion x [km]')
117                 ax.set_ylabel('Densidad rho [veh/km]')
118                 ax.set_title(titulo + '\n(-- -- solucion exacta)')
119                 ax.legend(fontsize=8.5)
120                 ax.set_xlim([0, L])
121                 ax.grid(True, alpha=0.25, ls='--')
122
123         plt.tight_layout()
124         plt.savefig('fig_s4_ej2_onda_trafico.png', dpi=300,
125 bbox_inches='tight')
126         plt.show()
127         print("Figura guardada.")

```

Listing 28: Semana 4 – Código 2: Ecuación de transporte 1D — comparación upwind vs Lax-Wendroff. Tráfico vehicular monitoreado por UAV. Ejecutar en Google Colab.

4.12.3. Código 3 — Ecuación de Poisson 2D: losa de concreto

```

1         """
2         Semana 4 - Ejemplo 3: Ecuacion de Poisson 2D (estado
3         estacionario)
4         Aplicacion: Distribucion de temperatura en losa de
5         concreto

```

```

4         con calor de hidratacion (Lerma Lomeli / Lerma
Martinez)
5
6         Ecuacion:  $d^2T/dx^2 + d^2T/dy^2 = -Q(x,y)/k$ 
7         Metodo: diferencias finitas centradas + scipy sparse
solver
8         """
9
10        import numpy as np
11        import matplotlib.pyplot as plt
12        from scipy.sparse import lil_matrix
13        from scipy.sparse.linalg import spsolve
14
15        #
16        =====
17        # BLOQUE 1: Geometria de la losa y parametros
fisicos
18        # Modifica Nx y Ny para obtener mayor resolucio
n
19        #
20        =====
21        Lx = 4.0    # [m] Longitud de la losa en x
22        Ly = 3.0    # [m] Ancho de la losa en y
23        Nx = 40     # Nodos en x (incrementar para mayor
precision)
24        Ny = 30     # Nodos en y
25
26        dx = Lx / (Nx - 1)
27        dy = Ly / (Ny - 1)
28        x_vec = np.linspace(0, Lx, Nx)
29        y_vec = np.linspace(0, Ly, Ny)
30        X, Y = np.meshgrid(x_vec, y_vec) # Malla 2D
31
32        N_total = Nx * Ny # Numero total de nodos
33        print(f"Sistema de {N_total} ecuaciones y {N_total}
incognitas")
34
35        #
36        =====
37        # BLOQUE 2: Funcion fuente (calor de hidratacion
gaussiana)
38        # Q_max: intensidad maxima del calor de hidratacion
[C/m2]

```



```

36         # sigma_f: radio de influencia de la fuente [m]
37         #
=====
38         x0_f    = Lx / 2.0    # Centro de la losa en x
39         y0_f    = Ly / 2.0    # Centro de la losa en y
40         sigma_f = 0.8         # Dispersión de la fuente [m]
41         Q_max   = 80.0        # Intensidad maxima [C/m2] (
normalizado, k=1)
42
43         # Evaluamos la fuente en todos los nodos de la malla
44         F = Q_max * np.exp(-(((X-x0_f)/sigma_f)**2 + ((Y-
y0_f)/sigma_f)**2))
45
46         #
=====
47         # BLOQUE 3: Funcion de conversion indice 2D ->
indice 1D
48         # Necesaria para mapear el nodo (i,j) a una fila del
sistema
49         #
=====
50         def idx(i, j):
51             """Convierte (fila_y=i, columna_x=j) a indice 1D."""
52             return i * Nx + j
53
54         #
=====
55         # BLOQUE 4: Ensamblaje del sistema lineal A*T = b
56         # Usamos matrices dispersas (lil_matrix) porque la
mayoria
57         # de los coeficientes son cero: solo 5 por fila
58         #
=====
59         A_mat = lil_matrix((N_total, N_total)) # Matriz
dispersa
60         b_vec = np.zeros(N_total)              # Vector
lado derecho
61
62         for i in range(Ny): # Recorre filas (direccion y)
63             for j in range(Nx): # Recorre columnas (direccion
x)
64                 n = idx(i, j)

```

```

65
66     # --- Nodos de frontera: condicion de Dirichlet ---
67     if i == 0 or i == Ny-1 or j == 0 or j == Nx-1:
68         # Temperatura prescrita segun posicion del borde
69         if i == 0:            T_bc = 20.0
# borde inferior
70         elif i == Ny-1:      T_bc = 25.0
# borde superior
71         else:                T_bc = 20.0 + 5.0*(i/(Ny-1))      #
    bordes laterales
72     A_mat[n, n] = 1.0
73     b_vec[n]    = T_bc
74
75     else:
76         # --- Nodos interiores: stencil de 5 puntos ---
77         # (T_derecha - 2*T_centro + T_izquierda) / dx^2
78         # + (T_arriba - 2*T_centro + T_abajo)    / dy^2
79         # = -Q(i, j)
80     A_mat[n, n]                = -2.0/dx**2 - 2.0/dy**2      #
diagonal
81     A_mat[n, idx(i, j+1)]    = 1.0/dx**2      # vecino
derecho
82     A_mat[n, idx(i, j-1)]    = 1.0/dx**2      # vecino
izquierdo
83     A_mat[n, idx(i+1, j)]    = 1.0/dy**2      # vecino arriba
84     A_mat[n, idx(i-1, j)]    = 1.0/dy**2      # vecino abajo
85     b_vec[n] = -F[i, j]      # termino fuente (con signo
negativo)
86
87     # Convertir a formato CSR (mas eficiente para la
resolucion)
88     A_csr = A_mat.tocsr()
89
90     #
=====
91     # BLOQUE 5: Resolucion del sistema lineal
92     # spsolve usa eliminacion directa eficiente para
matrices dispersas
93     #
=====
94     T_vec = spsolve(A_csr, b_vec)      # Resuelve A*T = b

```

```

95         T_campo = T_vec.reshape(Ny, Nx)      # Convierte a
        campo 2D

96
97         print(f"Temperatura maxima: {T_campo.max():.2f} C")
98         print(f"Temperatura minima: {T_campo.min():.2f} C")
99         print(f"Incremento por hidratacion: {T_campo.max() -
        25:.2f} C")

100
101         #
        =====
102         # BLOQUE 6: Visualizacion del campo de temperatura
103         #
        =====

104         fig, axes = plt.subplots(1, 2, figsize=(13, 5))
105
106         # Panel izquierdo: mapa de calor (contourf) con
        isolineas

107         cf = axes[0].contourf(X, Y, T_campo, levels=25, cmap
        ='inferno')
108         cs = axes[0].contour(X, Y, T_campo, levels=10,
109         colors='white', linewidths=0.6, alpha=0.5)
110         axes[0].clabel(cs, fmt='%.0f C', fontsize=7)
111         plt.colorbar(cf, ax=axes[0]).set_label('Temperatura
        T [C]')

112         axes[0].set_xlabel('x [m]')
113         axes[0].set_ylabel('y [m]')
114         axes[0].set_title('Campo de temperatura T(x,y) -
        Losa de concreto')
115         axes[0].set_aspect('equal')

116
117         # Panel derecho: perfil T(x) en el eje central y =
        Ly/2

118         eje_central = Ny // 2
119         T_lineal = 20.0 + 5.0 * (eje_central/(Ny-1)) * np.
        ones(Nx)      # sin fuente

120         axes[1].plot(x_vec, T_campo[eje_central, :], 'r-',
        lw=2.0,

121         label=f'Con calor de hidratacion')
122         axes[1].plot(x_vec, T_lineal, 'b--', lw=1.5,
123         label='Sin fuente (solo CC)')
124         axes[1].fill_between(x_vec, T_campo[eje_central,:],
        T_lineal,
    
```

```

125         alpha=0.15, color='red')
126         axes[1].set_xlabel('x [m]')
127         axes[1].set_ylabel('Temperatura T [C]')
128         axes[1].set_title(f'Perfil en eje central (y = {
y_vec[eje_central]:.1f} m)')
129         axes[1].legend()
130         axes[1].grid(True, alpha=0.3, ls='--')
131
132         plt.tight_layout()
133         plt.savefig('fig_s4_ej3_losa.png', dpi=300,
bbox_inches='tight')
134         plt.show()
135         print("Figura guardada.")
136

```

Listing 29: Semana 4 – Código 3: Ecuación de Poisson 2D (sistema lineal disperso). Losa de concreto con calor de hidratación. Ejecutar en Google Colab.

4.12.4. Código 4 — Convección-difusión 1D y número de Péclet

```

1      """
2      Semana 4 - Ejemplo 4: Ecuacion de conveccion-
difusion 1D
3      Aplicacion: Transporte de marcador en bomba
centrifuga (Zavala Morales)
4
5       $dc/dt + u * dc/dx = D * d^2c/dx^2$ 
6      Se comparan tres regimenes: Pe bajo (difusion), Pe
medio, Pe alto (conveccion)
7      """
8
9      import numpy as np
10     import matplotlib.pyplot as plt
11
12     #
=====
13     # BLOQUE 1: Parametros del dominio y la velocidad
del fluido
14     #
=====
15     L      = 0.5      # [m] Longitud del canal (
seccion de la bomba)

```

```

16         Nx      = 200          # Numero de nodos espaciales
17         dx      = L / (Nx - 1)
18         x       = np.linspace(0, L, Nx)
19
20         u_conv   = 2.0          # [m/s] Velocidad de conveccion
    del fluido
21         t_total  = 0.15         # [s] Tiempo total de
    simulacion
22
23         #
    =====
24         # BLOQUE 2: Funcion de simulacion para un D dado
25         # Recibe el coeficiente de difusion y devuelve la
    evolucion
26         #
    =====
27         def simular(D, etiqueta):
28             """
29             Simula la conveccion-difusion 1D para un D dado.
30             Esquema: upwind para conveccion + centrado para
    difusion.
31             """
32             Pe = u_conv * L / D    # Numero de Peclet
33             print(f"{etiqueta}: Pe = {Pe:.1f}, D = {D:.5f} m2/s"
    )
34
35             # Paso de tiempo: limite mas restrictivo entre CFL y
    Fourier
36             dt_cfl   = dx / u_conv          # Limite convectivo
    (CFL)
37             dt_difus = dx**2 / (2.0 * D)    # Limite difusivo (
    Fourier)
38             dt       = 0.4 * min(dt_cfl, dt_difus) # Margen de
    seguridad
39             Nt       = int(t_total / dt) + 1
40
41             # Condicion inicial: pulso gaussiano de
    concentracion
42             x0_iny   = 0.04    # [m] Posicion de inyeccion
43             sigma    = 0.025   # [m] Anchura del pulso
44             c = np.exp(-((x - x0_iny)**2) / (2*sigma**2))
45

```

```

46         # Integracion temporal
47         tiempos_g = np.linspace(0, t_total, 5)
48         perfiles = [c.copy()]
49         t_vec     = [0.0]
50         t = 0.0
51
52         for _ in range(Nt):
53             c_new = c.copy()
54
55             # Termino convectivo: upwind (diferencia hacia atras
, u > 0)
56             # Representa que el fluido "trae" la concentracion
desde la izquierda
57             conv = u_conv * (c[1:-1] - c[:-2]) / dx
58
59             # Termino difusivo: diferencia centrada de segundo
orden
60             # Representa que el gradiente de concentracion
genera flujo de masa
61             diff = D * (c[2:] - 2.0*c[1:-1] + c[:-2]) / dx**2
62
63             c_new[1:-1] = c[1:-1] + dt * (-conv + diff)
64
65             # Condiciones de frontera: gradiente nulo (Neumann)
en ambos extremos
66             c_new[0] = c_new[1]
67             c_new[-1] = c_new[-2]
68             c_new = np.maximum(c_new, 0.0) # concentracion
>= 0
69
70             c = c_new.copy()
71             t += dt
72
73             for t_g in tiempos_g[1:]:
74                 if abs(t - t_g) < dt/2.0 and t_g not in t_vec:
75                     perfiles.append(c.copy())
76                     t_vec.append(t)
77
78             return x, np.array(t_vec), perfiles, Pe
79
80         #
=====

```

```

81         # BLOQUE 3: Simulacion para tres valores de D
82         # Pe = u*L/D: D grande -> Pe pequeno (difusion
dominante)
83         #
=====
84         D_1 = u_conv * L / 2.0      # Pe = 2    (difusion muy
activa)
85         D_2 = u_conv * L / 20.0     # Pe = 20   (regimen mixto)
86         D_3 = u_conv * L / 200.0    # Pe = 200  (conveccion
dominante)
87
88         _, t_1, p_1, Pe_1 = simular(D_1, "Pe bajo")
89         _, t_2, p_2, Pe_2 = simular(D_2, "Pe medio")
90         _, t_3, p_3, Pe_3 = simular(D_3, "Pe alto")
91
92         #
=====
93         # BLOQUE 4: Comparacion de los tres regimenes al
tiempo final
94         #
=====
95         fig, ax = plt.subplots(figsize=(10, 5))
96
97         ax.plot(x * 100, p_1[-1], color='teal',      lw
=2.2,
98         label=f'Pe = {Pe_1:.0f} - difusion dominante')
99         ax.plot(x * 100, p_2[-1], color='darkorange', lw
=2.2,
100        label=f'Pe = {Pe_2:.0f} - regimen mixto')
101        ax.plot(x * 100, p_3[-1], color='#CC0000',    lw
=2.2,
102        label=f'Pe = {Pe_3:.0f} - conveccion dominante')
103
104        ax.set_xlabel('Posicion x [cm]')
105        ax.set_ylabel('Concentracion c [a.u.]')
106        ax.set_title('Conveccion-Difusion: perfiles al
tiempo final\n'
107        f'(t = {t_total*1e3:.0f} ms, u = {u_conv} m/s)')
108        ax.legend(fontsize=10)
109        ax.set_xlim([0, L * 100])
110        ax.grid(True, alpha=0.3, ls='--')
111

```

```

112         plt.tight_layout()
113         plt.savefig('fig_s4_ej4_convdiff_bomba.png', dpi=300,
114             bbox_inches='tight')
115         plt.show()
116         print("Figura guardada.")
117         # Para ver la evolucion temporal de un solo regimen,
118         # descomenta el siguiente bloque y modifica '
119         perfiles' y 't_vec':
120         # for idx, (t_g, perfil) in enumerate(zip(t_2, p_2))
121         :
122         #         plt.plot(x*100, perfil, label=f't = {t_g*1e3
123         :.1f} ms')

```

Listing 30: Semana 4 – Código 4: Ecuación de convección-difusión 1D. Análisis del número de Péclet. Bomba centrífuga. Ejecutar en Google Colab.

4.13. Ejercicios Reto — Tarea Semana 4

Instrucciones generales. Los ejercicios están clasificados por nivel. Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 es optativo y tiene valor de punto extra. En todos los casos se requiere: (a) formulación matemática explícita de la EDP y sus condiciones de frontera; (b) código ejecutable en Google Colab; (c) mínimo una gráfica con ejes etiquetados, unidades y título; y (d) interpretación física de los resultados en no menos de tres observaciones cuantitativas.

4.13.1. Ejercicio 1 — Análisis de la estabilidad FTCS (Nivel intermedio)

Modifica el Código 1 para explorar el efecto del número de Fourier en la estabilidad:

- Ejecuta la simulación con $F_o = 0.45$ (estable, cerca del límite), $F_o = 0.50$ (límite exacto) y $F_o = 0.55$ (inestable). Grafica los tres campos $T(x, t)$ en la misma figura al tiempo $t = 0.05$ s.
- Calcula la temperatura máxima en el dominio para cada caso. ¿En qué instante se rompe la estabilidad para $F_o = 0.55$? Repórtalo numéricamente.
- Argumenta en una o dos frases por qué el esquema explícito FTCS falla para $F_o > 0.5$ (pista: análisis de von Neumann, Semana 3).

Rúbrica.

- Tres figuras correctas, valor de F_o crítico identificado con precisión temporal, justificación física completa.

- 2 Dos figuras correctas, el caso inestable graficado pero sin análisis temporal.
- 1 Una figura correcta, sin análisis de estabilidad.
- 0 Código no ejecuta o resultado sin coherencia física.

4.13.2. Ejercicio 2 — Condición de Robin y convección superficial (Nivel intermedio)

Modifica el Código 1 para reemplazar la condición de Dirichlet en $x = L$ por una condición de Robin:

$$-k \frac{\partial T}{\partial x} \Big|_{x=L} = h (T(L, t) - T_{\infty}), \quad (150)$$

con $h = 500 \text{ W}/(\text{m}^2 \text{ K})$ (convección natural fuerte) y $T_{\infty} = 30^\circ \text{C}$.

Implementación sugerida. La condición de Robin se discretiza en el último nodo como:

$$\frac{T_N^{n+1} - T_{N-1}^{n+1}}{\Delta x} = -\frac{h}{k} (T_N^{n+1} - T_{\infty}),$$

despejando T_N^{n+1} explícitamente.

- (a) Compara la historia térmica en el centro del cable con la condición de Dirichlet original vs la condición de Robin.
- (b) Calcula la temperatura de la superficie ($x = L$) para $t = 0.1 \text{ s}$ en ambos casos.
- (c) ¿Cuál condición es más conservadora para evaluar el riesgo térmico de la fibra óptica?

Rúbrica.

- 3 Implementación de Robin correcta, comparación cuantitativa de las dos condiciones, conclusión justificada.
- 2 Implementación correcta pero sin comparación cuantitativa.
- 1 Código incompleto pero con la ecuación de Robin correctamente planteada.

4.13.3. Ejercicio 3 — Difusión-reacción: carbonatación de concreto (Nivel avanzado, punto extra)

La carbonatación del concreto es un proceso de difusión del CO_2 atmosférico seguido de reacción química con la portlandita $[\text{Ca}(\text{OH})_2]$. En el modelo simplificado de Papadakis, la concentración de CO_2 en la profundidad x satisface:

$$\frac{\partial c}{\partial t} = D_{\text{CO}_2} \frac{\partial^2 c}{\partial x^2} - k_r c c_p(x), \quad (151)$$

donde $k_r = 5 \times 10^{-4} \text{ m}^3/(\text{mol} \cdot \text{s})$ es la constante de reacción y c_p la concentración de portlandita.

- (a) Implementa la Ec. 151 en 1D para una losa de 0.3 m de espesor, con $D_{CO_2} = 3 \times 10^{-10} \text{ m}^2/\text{s}$, $c_p = 300 \text{ mol/m}^3$ (uniforme inicialmente) y condición de frontera $c(0, t) = 0.014 \text{ mol/m}^3$ (concentración atmosférica de CO_2).
- (b) Grafica la concentración de CO_2 a $t \in \{1, 5, 10, 30\}$ años.
- (c) Estima la profundidad de carbonatación (frente donde c desciende al 10 % de su valor en la frontera) en función del tiempo y compara con la ley $x_f \propto \sqrt{t}$ de Fick.

Rúbrica.

- 3 Simulación correcta, curva $x_f(t)$ graficada y comparada con la ley de Fick, discusión de la desviación debida a la reacción.
- 2 Simulación correcta, frente identificado pero sin comparación analítica.
- 1 Código con la estructura correcta pero resultados inconsistentes (p.ej., concentración negativa sin corrección).